



UNIVERSITA' DEGLI STUDI DI CAGLIARI

FACOLTÀ DI SCIENZE

Corso di Laurea Magistrale in Informatica

Data Mining: Report Finale

Gruppo G3.3

Componenti:

Aresu Matteo

60/99/00014

Mantega Gabriele

60/99/00006

Masala Gabriele

60/99/00004

ANNO ACCADEMICO 2025-2026

1 Introduzione

Questo report affronta il problema della classificazione dei documenti testuali, applicato all'analisi delle interazioni interne dei newsgroup, spazi online storicamente strutturati su infrastrutture decentralizzate, e dedicati al dibattito su topic ben determinati. L'obiettivo centrale del progetto è l'utilizzo di diversi algoritmi di classificazione, spaziando dai modelli storici tradizionali (shallow learning) ai modelli a combinazione (ensemble learning), al fine di predire il topic di un post a partire dal suo contenuto. Lo studio non si è limitato all'analisi delle prestazioni dei classificatori, ma si estende alla comparazione dei metodi di rappresentazione geometrica dei dati in input. A questo scopo sono stati utilizzati due metodi di vettorizzazione: **TF-IDF** (Term Frequency-Inverse Document Frequency) e **GloVe** (Global Vectors for Word Representation). Il progetto è stato realizzato con l'ausilio dell'infrastruttura di Jupyter, il linguaggio di programmazione Python e delle sue librerie fondamentali, come pandas, scikit-learn, NumPy, Matplotlib. Il presente report è organizzato nei seguenti passi:

- **Sezione 2 - Dataset** Analisi del dataset 20newsgroups, della sua struttura e delle sue categorie tematiche con approfondimento su quelle individuate per il task.
- **Sezione 3 - Preprocessing** Descrizione del flusso di elaborazione e trasformazione dei dati, differenziando i due metodi di vettorizzazione scelti.
- **Sezione 4 - Applicazione modelli** Elenco dei classificatori utilizzati, con relative descrizioni ed esposizione sulle scelte di configurazione effettuate sugli iperparametri.
- **Sezione 5 - Discussione** Esposizione dei risultati ottenuti a seguito di training e testing, con considerazioni argomentate e rappresentazione visiva dei dati, mediante tabelle e grafici informativi.

Gruppo di Discussione	Descrizione
<code>rec.autos</code>	Discussioni su motori, recensioni, manutenzione e novità automobilistiche.
<code>rec.sport.baseball</code>	Spazio per appassionati di baseball: analisi, statistiche e campionati.
<code>sci.electronics</code>	Forum su progettazione di circuiti, componenti e teoria elettronica.
<code>sci.med</code>	Discussioni su medicina, salute, ricerche scientifiche e trattamenti.
<code>sci.space</code>	Spazio dedicato all'esplorazione spaziale, astronomia e tecnologia dei razzi.

Tabella 2.1: Elenco dei newsgroup utilizzati nel progetto e relative descrizioni

2 Dataset

Il dataset di riferimento utilizzato per la sperimentazione è **20newsgroups**, un corpus di circa 18.000 pubblicazioni ampiamente utilizzata nel Natural Language Processing (NLP), e distribuite all'interno di venti differenti categorie tematiche. Una caratteristica di questo dataset è l'organizzazione in forma gerarchica dei suoi topic, formalizzata dalla seguente struttura: `macrocategoria.sottocategoria` (con la possibilità di inserire ulteriori sottocategorie, successivamente alla prima). La documentazione relativa e i dati di partenza sono consultabili liberamente al seguente link: <https://www.kaggle.com/datasets/crawford/20-newsgroups>.

Ai fini dell'elaborazione del dataset, è stato utilizzato solo un sottoinsieme di cinque categorie per analizzare il comportamento dei classificatori, di cui alcune rappresentanti tematiche potenzialmente sovrapponibili concettualmente. Le categorie utilizzate in questo scopo sono riportate in Tabella 2.1. Dal punto di vista strutturale, ogni singolo documento del dataset corrisponde a un post creato da un utente. In linea con gli standard Usenet, ogni post è strutturato nel seguente modo:

- **Header:** una sezione iniziale del documento, dove vengono inseriti alcuni metadati tecnici, come l'indirizzo email del mittente, il server di provenienza e l'oggetto del messaggio.
- **Body:** il testo effettivo del post scritto dall'utente, un insieme di frasi in linguaggio naturale costituenti il vero nucleo informativo.
- **Footer:** una sezione finale opzionale dedicata alle firme automatiche o citazioni personali.

Un post di esempio, che rispetta questa struttura, è osservabile in Figura 2.1. Il dataset è stato caricato in locale tramite la funzione `datasets.fetch_20newsgroups()` di scikit-learn ed è stato successivamente ripartito in **training set** e **test set**.

In Figura 2.2 è rappresentata la suddivisione del dataset. Nello specifico in Figura 2.2a è rappresentata la distribuzione dei dati tra gli split di train e di test, mentre in Figura 2.2b viene riportata la distribuzione delle classi all'interno di training e test set, tenendo presente che queste rappresentano i gruppi di discussione. Si noti che le classi sono distribuite in maniera omogenea all'interno del dataset. Data l'assenza di sbilanciamenti nel dataset siamo in grado di comparare più equamente le prestazioni tra le diverse classi.

From: baalke@kelvin.jpl.nasa.gov (Ron Baalke)
Subject: Galileo Update - 04/15/93
Organization: Jet Propulsion Laboratory
Lines: 113
Distribution: world
NNTP-Posting-Host: kelvin.jpl.nasa.gov
Keywords: Galileo, JPL
News-Software: VAX/VMS TNEWS 1.41

Forwarded from Neal Ausman, Galileo Mission Director

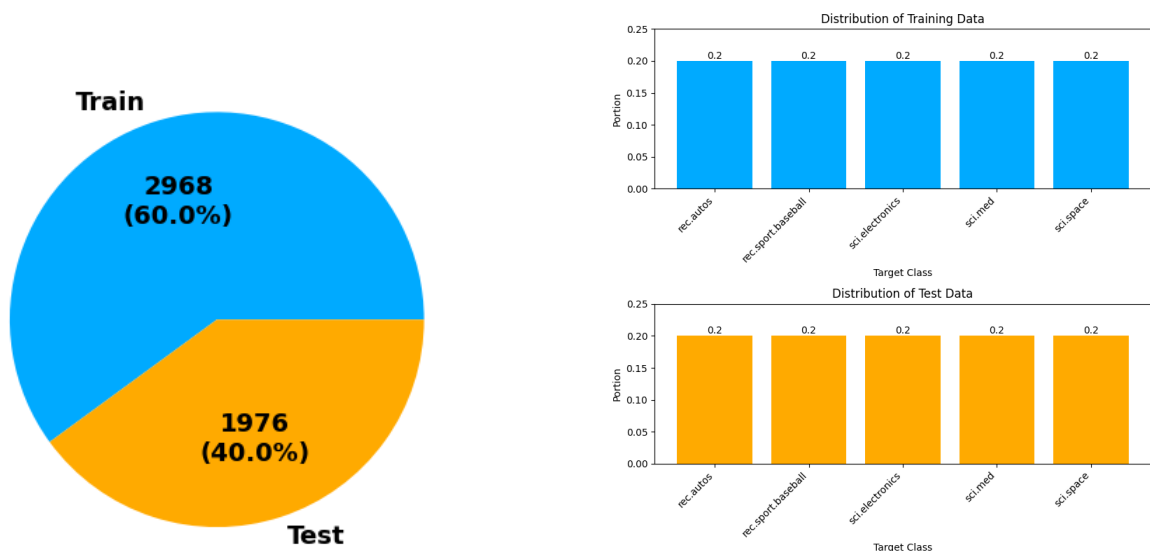
GALILEO
MISSION DIRECTOR STATUS REPORT
POST-LAUNCH
April 9 - 15, 1993

SPACECRAFT

1. On April 9, real-time commands were sent, as planned, to reacquire celestial reference completion of the Low Gain Antenna (LGA-2) swing/Dual Drive Actuator (DDA) hammer activities.

. . .

Figura 2.1: Estratto di un post del topic sci.space.



(a) Distribuzione degli oggetti tra training set e test set.

(b) Distribuzione omogenea delle classi nel dataset in esame (Train set in alto, Test set in basso).

Figura 2.2: Grafici relativi distribuzione dei dati nel dataset.

3 Preprocessing

La fase di preprocessing è finalizzata alla conversione dei dati testuali grezzi, contenuti nei documenti del dataset, in rappresentazioni numeriche, tramite un processo chiamato vettorizzazione. In questo progetto sono stati implementati e confrontati due approcci metodologici: la rappresentazione frequenziale **TF-IDF** e l'utilizzo di vettori semantici densi tramite **embeddings GloVe**. Entrambe le metodologie sono state configurate per lavorare su uno spazio lineare a cento dimensioni, come da specifiche.

- **Vettorizzazione TF-IDF**

Il metodo TF-IDF (Term Frequency-Inverse Document Frequency) costruisce un dizionario interno al modello, inserendo ogni parola presente nel dataset. Per ogni parola del dizionario, viene calcolato un punteggio TF-IDF. Il nucleo dell'algoritmo prevede il calcolo della frequenza di una parola all'interno del singolo post (frequenza intra-documento) pesata rispetto all'inverso della sua diffusione nell'intero dataset (frequenza inter-documento). L'intuizione è quella di restituire un valore TF-IDF più alto alle parole rare, in quanto in possesso di elevato potere discriminante per l'identificazione del topic di appartenenza dell'intero messaggio. In Equazione (1) è riportata la formula utilizzata da `TfidfVectorizer()`, implementato da scikit-learn, per il calcolo del punteggio TF-IDF per un dato termine t all'interno di un documento d . Si noti che i parametri di *default* prevedono lo *smoothing* aggiungendo 1 per evitare la divisione per zero.

$$\begin{aligned} \text{TF-IDF} &= \text{tf}(t, d) \times \text{idf}(t) \\ \text{idf}(t) &= \log \left(\frac{1 + N}{1 + \text{df}(t)} \right) + 1 \end{aligned} \tag{1}$$

- **Embedding Semantici GloVe**

A differenza dell'approccio statistico di TF-IDF, GloVe si appoggia ad un dizionario di vettori generato da un modello appositamente addestrato. Ogni parola viene proiettata su uno spazio geometrico continuo a N dimensioni, in cui le coordinate precise di un termine sono in grado di rappresentare il suo significato semantico. Questa rappresentazione ha il vantaggio che, parole con significato semantico simile, risulteranno vicine geometricamente secondo metriche di similarità come il coseno o la distanza euclidea. Ciò permette ai classificatori di intercettare relazioni tra parole sinonime e contesti logici che il semplice calcolo delle frequenze non sarebbe in grado di scoprire.

In entrambi i casi, prima della vettorizzazione, viene eseguito un passo di normalizzazione del testo e di pulizia dalle stop-words. Queste sono parole di uso comune in un linguaggio che non trasmettono un significato semantico rilevante, che tuttavia fanno parte della grammatica. Un esempio di tali parole sono gli articoli e le preposizioni, presenti sia nella lingua italiana che in quella inglese. Nel caso di TF-IDF, questa fase di pulizia avviene internamente alla funzione `TfidfVectorizer()` impostando il parametro `stop_words="english"`. Nel caso di GloVe, abbiamo implementato le seguenti funzioni:

- `preprocess_text(text)`: esegue il *lowercasing*, elimina caratteri speciali e punteggiatura, rimuove le stop-words (ovvero le occorrenza di `sktext.ENGLISH_STOP_WORDS` all'interno del documento) ed effettua la *tokenization* delle parole;
- `load_embeddings(file_path)`: carica il dizionario degli embeddings a partire dal file `glove.6B.100d.txt`, che presenta al suo interno un vettore di 100 valori numerici per ciascuna parola;
- `get_document_embedding(tokens, glove_embeddings)`: per ciascun token di un documento, ottiene l'embedding corrispondente, se presente nel dizionario GloVe, e calcola la media di tutti questi embedding per ottenere un unico vettore che rappresenta il documento. Se nessun token è presente all'interno del dizionario, il document embedding corrispondente è il vettore nullo.

4 Applicazione Modelli

La fase sperimentale ha previsto l'addestramento di sei modelli di classificazione multi-classe. Di seguito si propone, per ciascun algoritmo, una breve descrizione unita alla giustificazione metodologica della scelta dei relativi iperparametri.

- **Decision Tree (DT)**

Classificatore di shallow learning, basato su un albero decisionale in cui i nodi foglia rappresentano le classi e i nodi genitore rappresentano criteri decisionali. Nello specifico, la sua implementazione su scikit-learn avviene tramite albero binario. È stata impostata una profondità massima `max_depth=8`, che è piuttosto limitata, per evitare overfitting sul dataset. Una misura di profondità eccessivamente alta potrebbe infatti condurre l'albero a imparare i criteri esatti per la classificazione. Invece, in tale modo abbiamo al più $2^{(8-1)} = 128$ nodi decisionali, che si avvicinano al numero di features utilizzate. È stato impostato un limite inferiore al numero di campioni necessari in una classe per eseguire uno split. Il valore scelto è `min_samples_split=0.01`, corrispondente all'1% del dataset. Ciò garantisce una crescita più controllata dell'albero.

- **K-Nearest Neighbors (KNN)**

Ampiamente considerato uno dei più semplici algoritmi di machine learning. Secondo il KNN, un oggetto è classificato in base alla maggioranza dei voti dei suoi k vicini (con k un iperparametro intero positivo). La sua implementazione su scikit-learn non ha una vera fase di addestramento, si limita a inserire l'intero dataset sulla memoria centrale. Questa implementazione ha il vantaggio di avere un costo computazionale pari a $O(1)$, tuttavia risulta in pessime prestazioni con dataset molto grandi. La scelta degli iperparametri è stata guidata dalla natura intrinseca dei dati testuali vettorializzati. In primo luogo è stata adottata la metrica di similarità del coseno (`metric='cosine'`), alternativamente alla più classica distanza euclidea. Questa scelta deriva dal fatto che, usando rappresentazioni geometriche vettoriali (GloVe e TF-IDF), l'orientamento dei vettori cattura il significato semantico e concettuale in modo più robusto rispetto alla loro norma geometrica. Parallelamente, il numero di vicini è stato fissato a quindici (`n_neighbors=15`), per avere un bilanciamento ottimale tra bias e varianza. Un valore di K eccessivamente alto o eccessivamente basso potrebbe portare il modello a condizioni di, rispettivamente, overfitting o underfitting.

- **Multi Layer Perceptron (MLP)**

Modello di rete neurale artificiale, costruita con strati multipli di nodi in un grafo diretto, di cui ogni strato è direttamente connesso al successivo. Il suo obiettivo è mappare un insieme di dati di ingresso in un insieme di dati di uscita appropriati. L'architettura della rete neurale artificiale è stata modellata con una scelta degli iperparametri in grado di catturare le relazioni non lineari dei vettori testuali. Il numero di nodi di input scelto è pari a 64, ovvero la potenza di 2 più alta e inferiore di 100. Così facendo la rete viene forzata a comprimere le informazioni essenziali e ad scartare quelle poco salienti. Il numero di nodi dell'hidden layer contrae la dimensionalità a 32 neuroni, agendo come un ulteriore imbuto. Gli iperparametri scelti per l'input layer e l'hidden layer sono `hidden_layer_sizes=(64, 32)`. Per quanto riguarda il processo di ottimizzazione, il limite massimo delle epoche è stato elevato a trecento, per garantire un margine computazionale sufficiente a convergere verso un minimo locale stabile (`max_iter=300`). Al fine di prevenire fenomeni di overfitting, è stata attivata la regolarizzazione dinamica tramite `early_stopping=True`.

- **Random Forest (RF)**

Classificatore di ensemble learning, ottenuto dall'aggregazione tramite bagging di diversi Decision Tree, nasce come soluzione atta a ridurre l'overfitting del training set di quest'ultimi. Gli iperparametri scelti per gli estimatori sono stati quelli di default, poichè le loro prestazioni, lavorando parallelamente e con output congiunto, sono soddisfacenti. Inoltre, poichè il RF permette ai suoi estimatori di lavorare in parallelo e non sequenzialmente, abbiamo impostato come numero di quest'ultimi pari a 200, un numero più alto della media, ma comunque non sufficiente per l'overfitting (`n_estimators=200`).

- **Adaptive Boosting (AdaBoost, AB)**

L'intuizione che sta alla base dell'AdaBoost risiede nell'addestramento sequenziale di un insieme di classificatori deboli (weak learners). A differenza dei metodi di bagging (come, ad esempio, il RF), in cui i modelli operano in parallelo e con parità di potere decisionale, AdaBoost adotta un approccio in cui ogni nuovo stimatore viene addestrato focalizzandosi prevalentemente sui punti classificati erroneamente dagli scorsi stimatori. Per il classificatore AdaBoost, la scelta degli iperparametri è stata orientata alla ricerca di un equilibrio tra accuratezza della convergenza e robustezza rispetto all'overfitting. Il numero di estimatori deboli è stato impostato a 100 (`n_estimators=100`). Questo poichè AdaBoost utilizza come modelli di base dei decision stumps (DT con un solo livello di profondità), e quindi con potere decisionale limitato, focalizzando il lavoro complessivo sulle istanze testuali più complesse e più spesso classificate come errate. La scelta di voler inserire un learning rate ridotto a 0.5 (`learning_rate=0.5`) deriva da una regolarizzazione geometrica, che costringe il modello a essere lento nell'apprendimento. In questo modo si limita il raggiungimento dell'overfitting.

- **eXtreme Gradient Boosting (XGBoost, XGB)**

Evoluzione dell'AdaBoost, basato sul paradigma del Gradient Tree Boosting. La calibrazione degli iperparametri mira a bilanciare la capacità predittiva dell'insieme e la stabilità della convergenza, in modo simile al classificatore AdaBoost. Il numero di estimatori è stato impostato a 100 (`n_estimators=100`), un valore sufficiente a consentire all'algoritmo di minimizzare i residui della funzione di perdita del gradiente. In secondo luogo, il tasso di apprendimento è stato fissato a 0.5 (`learning_rate=0.5`), operando come fattore di contrazione e garantendo una convergenza più robusta.

Model	Accuracy	Precision	Recall	F1-score
Decision Tree	0.56	0.69	0.56	0.58
KNN	0.57	0.57	0.57	0.57
MLP	0.62	0.62	0.62	0.62
Random Forest	0.64	0.66	0.64	0.65
AdaBoost	0.57	0.68	0.57	0.58
XGBoost	0.63	0.64	0.63	0.63

(a) Risultati dei modelli con TF-IDF

Model	Accuracy	Precision	Recall	F1-score
Decision Tree	0.66	0.66	0.66	0.66
KNN	0.88	0.88	0.88	0.88
MLP	0.89	0.89	0.89	0.89
Random Forest	0.87	0.87	0.87	0.87
AdaBoost	0.80	0.81	0.80	0.81
XGBoost	0.88	0.88	0.88	0.88

(b) Risultati dei modelli con TF-IDF

Tabella 5.1: Metriche risultanti dal testing dei modelli addestrati

5 Discussione

Una volta configurati gli iperparametri, abbiamo eseguito l'addestramento e il testing di ciascun modello. In seguito al test sono state raccolte le metriche richieste: **Accuracy**, **Precision**, **Recall** ed **F1-score**. In Tabella 5.2 sono riportate le formule delle metriche utilizzate. F1-score serve lo scopo di media armonica tra Precision e Recall, andando a penalizzare il punteggio in direzione della metrica col valore più basso. È comunque interessante considerare i singoli valori di Precision e Recall, per conoscere rispettivamente: la percentuale di predizioni corrette sul totale delle stime svolte nei confronti di una classe; la percentuale di predizioni corrette sul totale di campioni di una data classe. Infine, l'accuratezza descrive in maniera più generale le prestazioni dei modelli attraverso la percentuale di predizioni corrette. Visto che il dataset si presenta bilanciato, questa metrica risulta essere tanto esplicativa quanto intuitiva. I valori delle metriche raccolti a seguito del testing sono riportati in Tabella 5.1 e distinti per TF-IDF (Tabella 5.1a) e GloVe (Tabella 5.1b). Recall e F1-score sono state calcolate con `averaging='macro'`, quindi come media aritmetica delle performance per-classe.

In Figura 5.1a viene mostrato il grafico delle prestazioni dei modelli attraverso l'uso di TF-IDF. I valori di accuratezza sono globalmente bassi, con media 0.60 ± 0.03 . I modelli ensemble mostrano prestazioni leggermente più alte, ma non superiori all'ordine di $+0.10$. L'unica eccezione tra gli ensemble è AdaBoost che mostra prestazioni ridotte, assimilabili al DT.

In Figura 5.1b viene mostrato il grafico delle prestazioni degli stessi modelli ma attraverso l'utilizzo di GloVe. Le prestazioni sono nettamente migliorate, passando ad una media 0.83 ± 0.08 . Anche in questo caso abbiamo prestazioni alte nei modelli ensemble, persino in AdaBoost che si dimostra comunque più debole degli altri classificatori. Si nota un importante miglioramento di KNN, che riporta le prestazioni più alte dopo MLP. Il modello peggiore rimane DT.

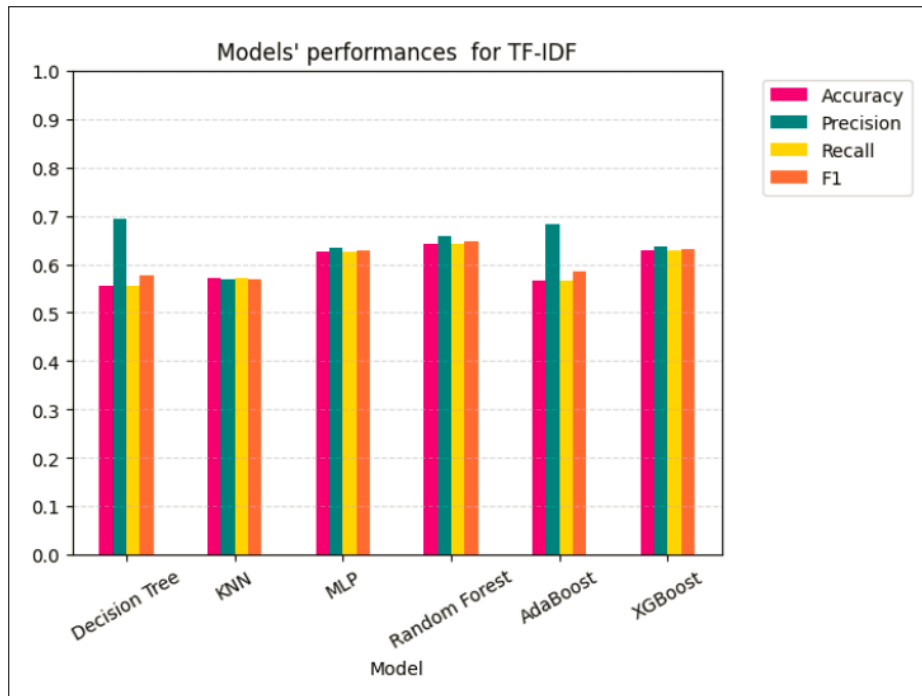
Metrica	Formula
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$
Precision	$\frac{TP}{TP + FP}$
Recall	$\frac{TP}{TP + FN}$
F1-Score	$\frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$
$TP = \text{True Positive}$ $FP = \text{False Positive}$ $TN = \text{True Negative}$ $FN = \text{False Negative}$	

Tabella 5.2: Riepilogo delle metriche di performance per la classificazione.

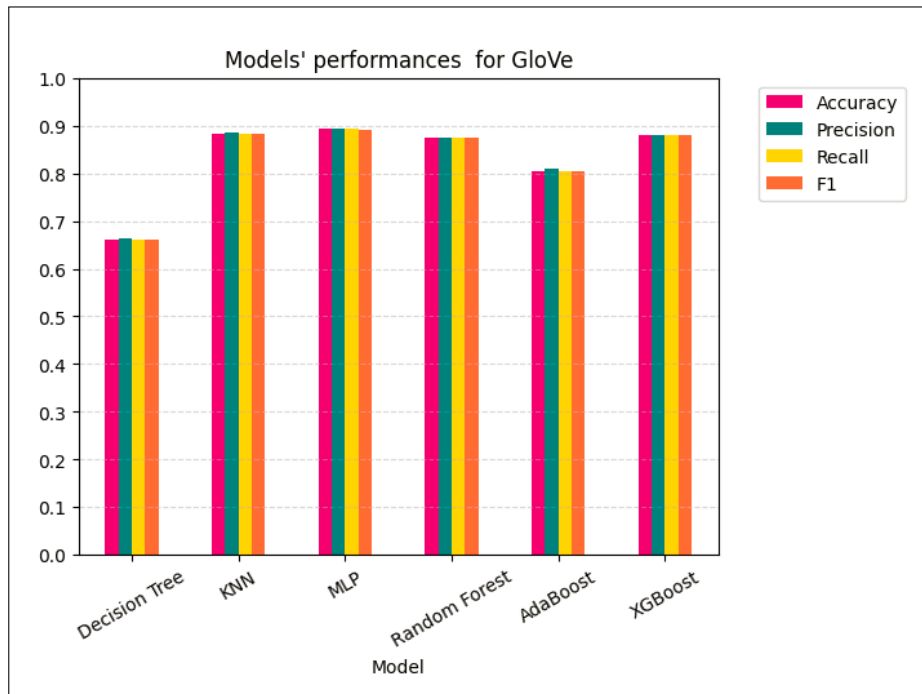
Ci soffermiamo a osservare KNN. Mentre con TF-IDF mostrava prestazioni basse, ora risulta paragonabile ai classificatori più complessi. Ipotizziamo che ciò dipenda direttamente dal principio su cui GloVe si fonda: rappresentare in maniera spaziale relazioni semantiche tra le parole. Infatti, KNN utilizza una metrica di distanza per eseguire il *voting* della classe di un certo elemento. Nel nostro caso andiamo ad utilizzare la metrica del coseno, adatta ai vettori, insieme al *weighting* del voting in base alla prossimità dei *neighbors* del punto considerato. Pensiamo che tutti questi fattori basati sulla distanza spaziale contribuiscano alle alte prestazioni di KNN. In questo contesto potrebbe essere considerato il modello migliore in quanto si presenta come un modello semplice, intuitivo ed efficiente, dato che non ha neanche bisogno di un vero tempo di addestramento come invece avviene negli altri modelli. Per quanto MLP mostri prestazioni leggermente più alte, perdoni rilievo se paragonate al tempo impiegato da KNN per svolgere addestramento e predizione dei dati: MLP richiede un vero addestramento suddiviso in epoche, mentre KNN ha solo bisogno del calcolo di N distanze, tante quante sono i neighbors impostati per il modello.

In Figura 5.2 sono riportate le matrici di confusione per ciascun modello e metodo di rappresentazione dei dati. Questa visualizzazione delle matrici consente di svolgere delle valutazioni qualitative sulle prestazioni dei modelli. In tutti i modelli con TF-IDF abbiamo un'evidente difficoltà nella predizione delle classi `sci.eletronics` e `sci.med`. In particolare, se esaminiamo DT e AB notiamo una polarizzazione rispettivamente verso `sci.med` e `sci.eletronics`. Non siamo in grado di individuare le motivazioni dal momento che i modelli come DT e AB generano automaticamente i criteri di identificazione delle classi. Possiamo ipotizzare che i modelli abbiano rilevato una caratteristica che avrebbe potuto essere distintiva per la classe polarizzata, ma che invece si presenta in tutte le classi. Ciò che invece possiamo notare dalle matrici di confusione di GloVe, oltre ad una diagonale con valori stabili, è una piccola relazione di somiglianza tra la classe `rec.autos` e `sci.eletronics`, nonché una leggera confusione tra le predizioni della categoria `sci`. Probabilmente ciò è dovuto dalla presenza di termini comuni, soprattutto all'interno della stessa categoria.

Infine, riportiamo in Figura 5.3 le metriche di Accuracy ed F1-score, calcolate per-classe. Il calcolo è avvenuto in modalità *One vs Rest*, considerando una classe per volta come la positiva e la totalità delle restanti come la negativa. In Figura 5.3a possiamo notare in maniera più



(a) TF-IDF



(b) GloVe

Figura 5.1: Metriche raccolte dal testing dei classificatori

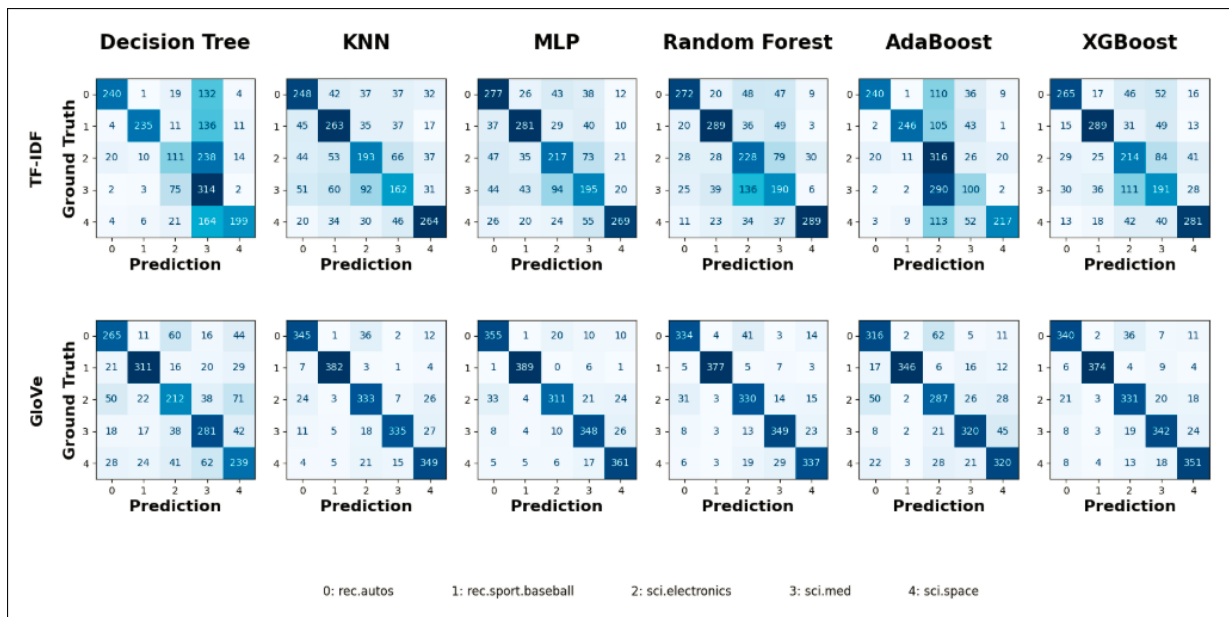
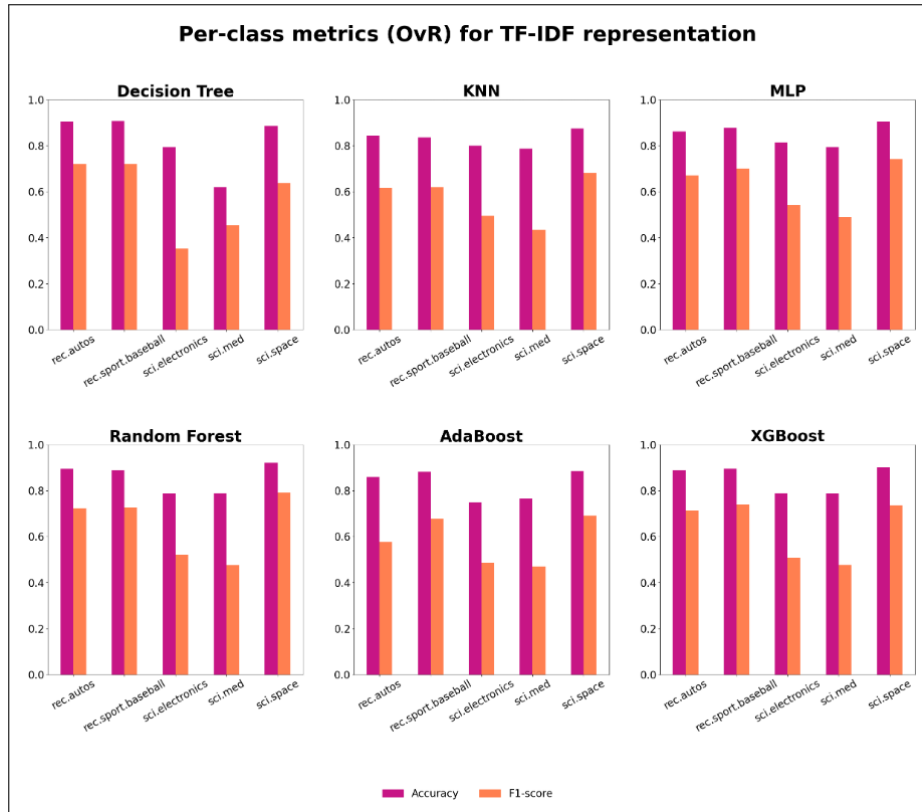


Figura 5.2: Matrici di confusione di ciascun modello e metodo di rappresentazione dei dati (TF-IDF in alto, GloVe in basso).

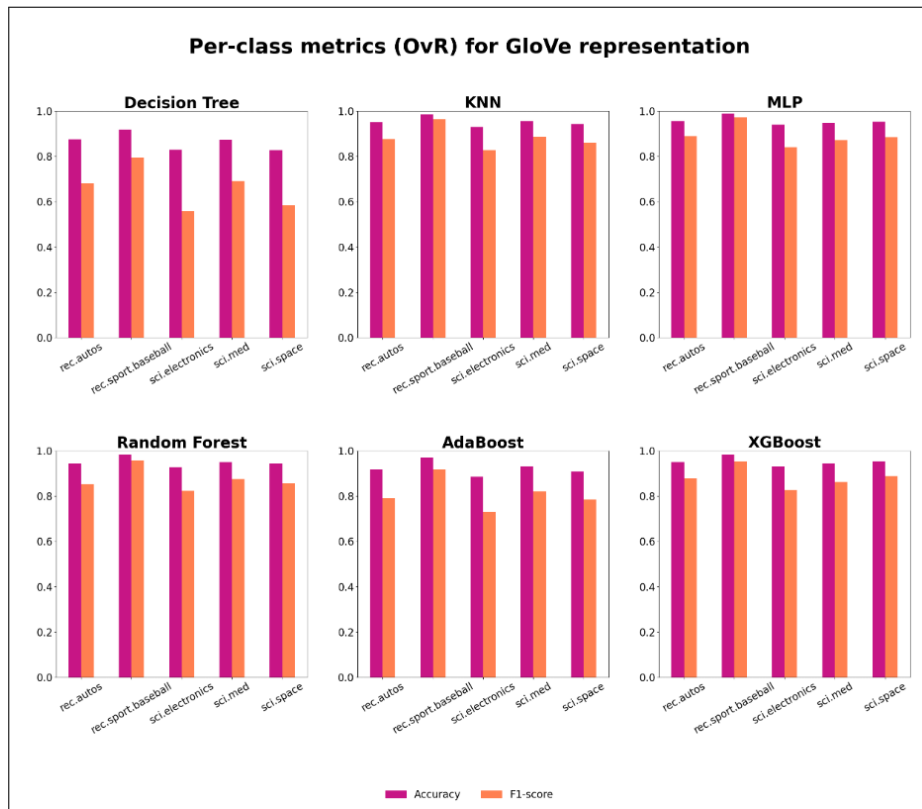
evidente le difficoltà di classificazione riportate precedentemente.

6 Conclusioni

In conclusione, abbiamo svolto training e testing di sei modelli di classificazione attraverso due metodi di vettorizzazione: TF-IDF e GloVe. Abbiamo valutato le prestazioni dei modelli attraverso le metriche raccolte che abbiamo anche visualizzato in forma di grafici a barre, riscontrando prestazioni basse per TF-IDF in confronto a GloVe. Possiamo quindi ipotizzare che l'utilizzo di Word Embeddigs favorisca i modelli nei compiti di classificazione per via della loro capacità di distribuire l'informazione semantica delle parole tra tutte le features dei vettori. Concludiamo che l'approccio degli embeddings vince contro l'approccio statistico-frequenziale in un task di classificazione di NLP, come in quello preso in esame. Inoltre, possiamo affermare che le prestazioni dei classificatori dipendono fortemente dai dati in input. Un buon preprocessing dei dati, che consenta di estrarne il più possibile la natura informativa può essere cruciale per la buona riuscita del task. KNN dimostra che modelli semplici e intuitivi possono essere messi al pari di modelli più complessi, se configurati correttamente e se associati ai giusti dati in ingresso.



(a) TF-IDF



(b) GloVe

Figure 5.3: Prestazioni per-classe (One vs Rest) dei modelli